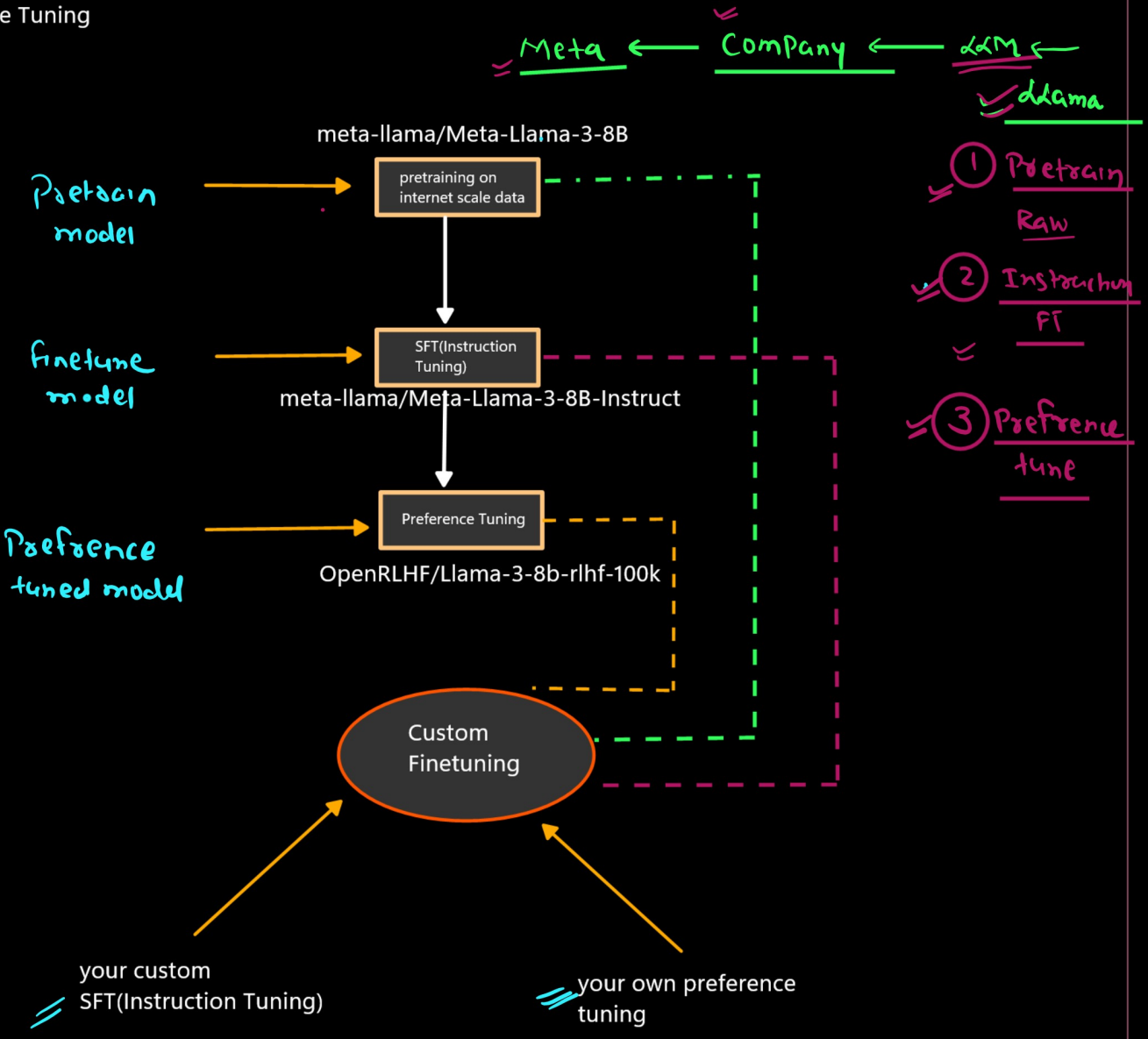
Fine-tuning

Fine-tuning is the process of taking a already trained model and training it further on domain-specific or task-specific data so that it performs better for a particular use case.

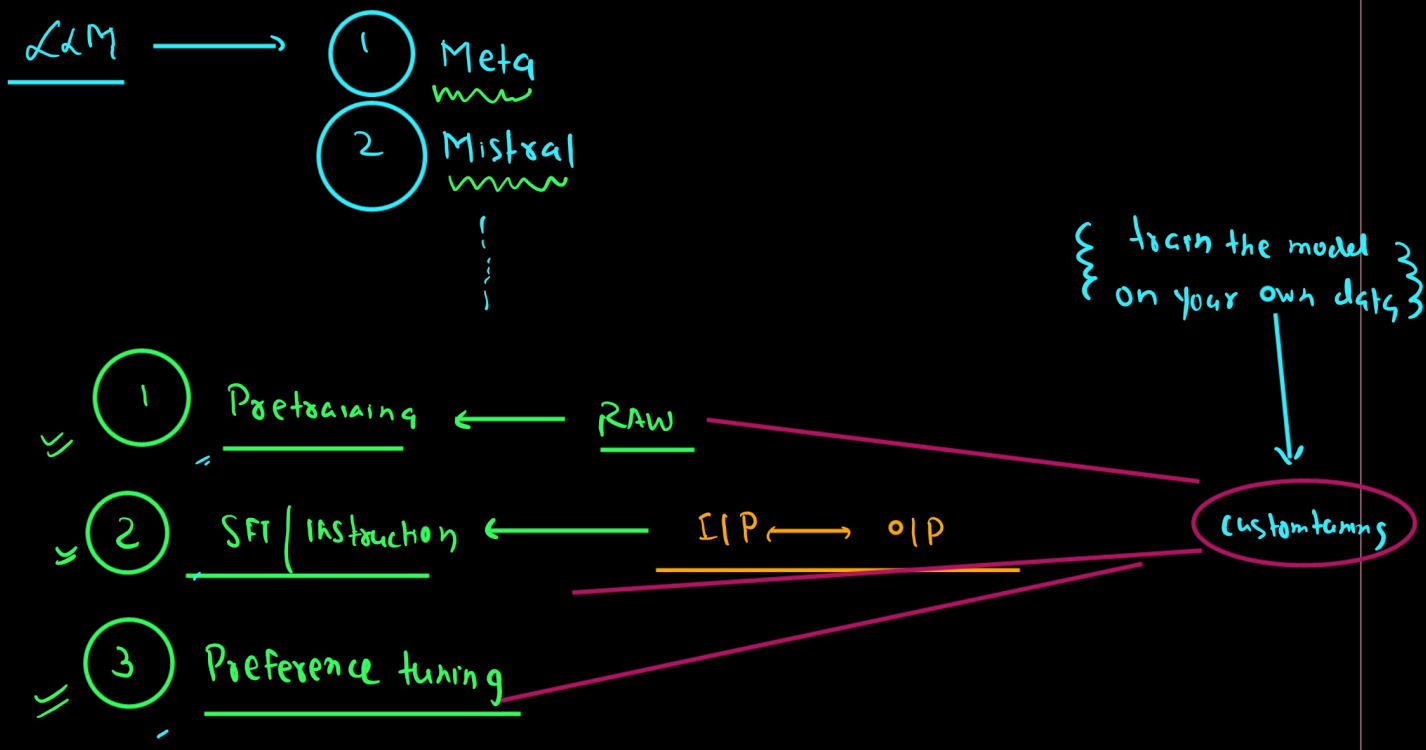
LLM Training Pipeline
1. Pretraining of model
2. Supervised Finetuning(Instruction Finetuning)
3. Preference Tuning



1. hugging face Modules
2. unsloth

X

1. FULL Finetuning
2. Partial Finetuning(PEFT)



meta-llama/Meta-Llama-3-8B

↓

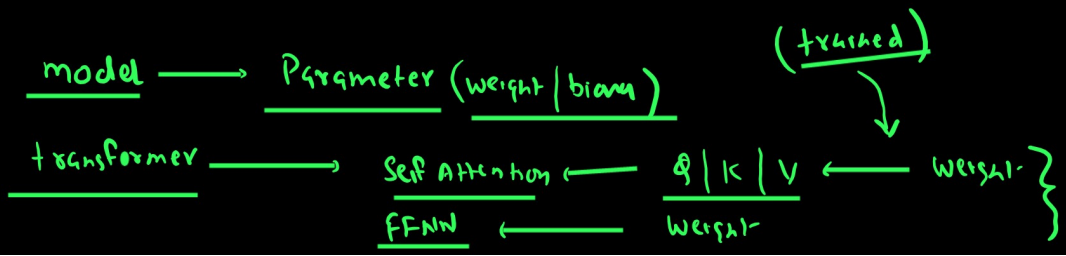
meta-llama/Meta-Llama-3-8B-Instruct
(how to follow the instruction)

usecase: Pharama --> document on molecule study
1. to train a llm from scratch it is difficult
2. i will take any opensource model i will download it on my owm server
3. and then i will finetune that

my requirement was just to generate the data not the conversation AI/Chatbot/QA

my requirement was just to generate the data
meta-llama/Meta-Llama-3-8B --> i have trained(finetuning) on my own pharama datset

conversation AI/Chatbot/QA
meta-llama/Meta-Llama-3-8B-Instruct --> further i will finetuning it on my own dataset
IP--OP



1. Pretraining (Unsupervised / Self supervised learning)
2. SFT / IFT → Supervised finetuning (I/P ⟷ O/P)

Parameter level

- Huge GPU Power, Huge infra
- Full finetuning ← Retrain all the parameter (weight & bias)
- Partial finetuning ← Subset of parameter

Two method

1. OLD school method
2. CV
   - CNN → VGG / Resnet ...
   - BERT
   - BART
   - T5

retrain

1. Freeze all the layer train only last O/P layer
2. Freeze some starting layer & retrain the remaining last layer

(Latest ← LLM ← Huge architecture)

Latest-

2. PEFT ← Small Infra

Parameter efficient FT

Single GPU | small VRAM

Low Rank Adaption

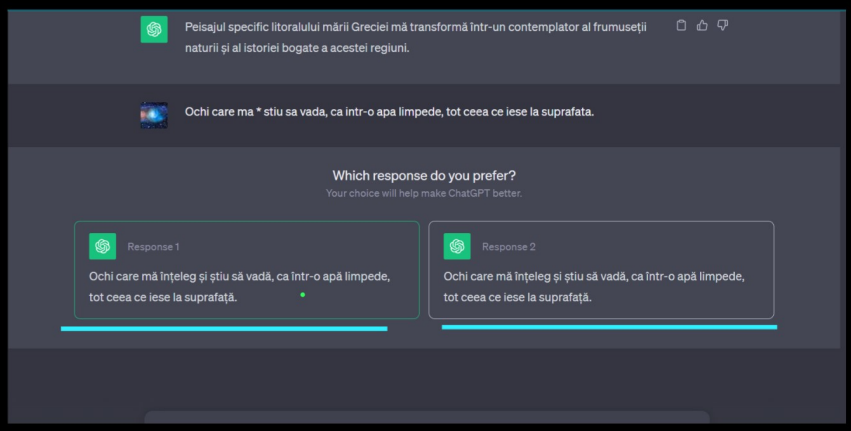
1. LoRA → QLoRA
   - ADAPTER
2. DoRA (weight-decomposition)

Quantized (Float → small float / int)

3. BitFit-
4. IA^3

X

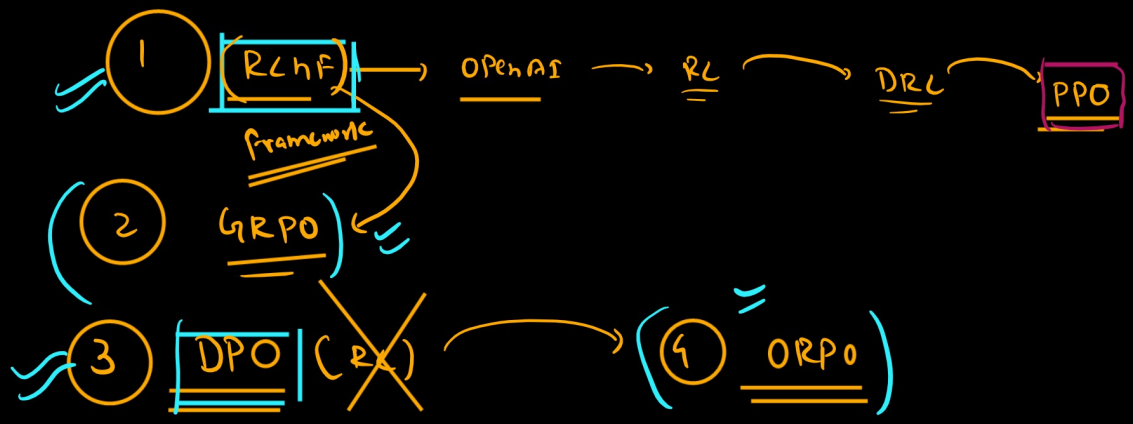
③ Prefrence alignment-

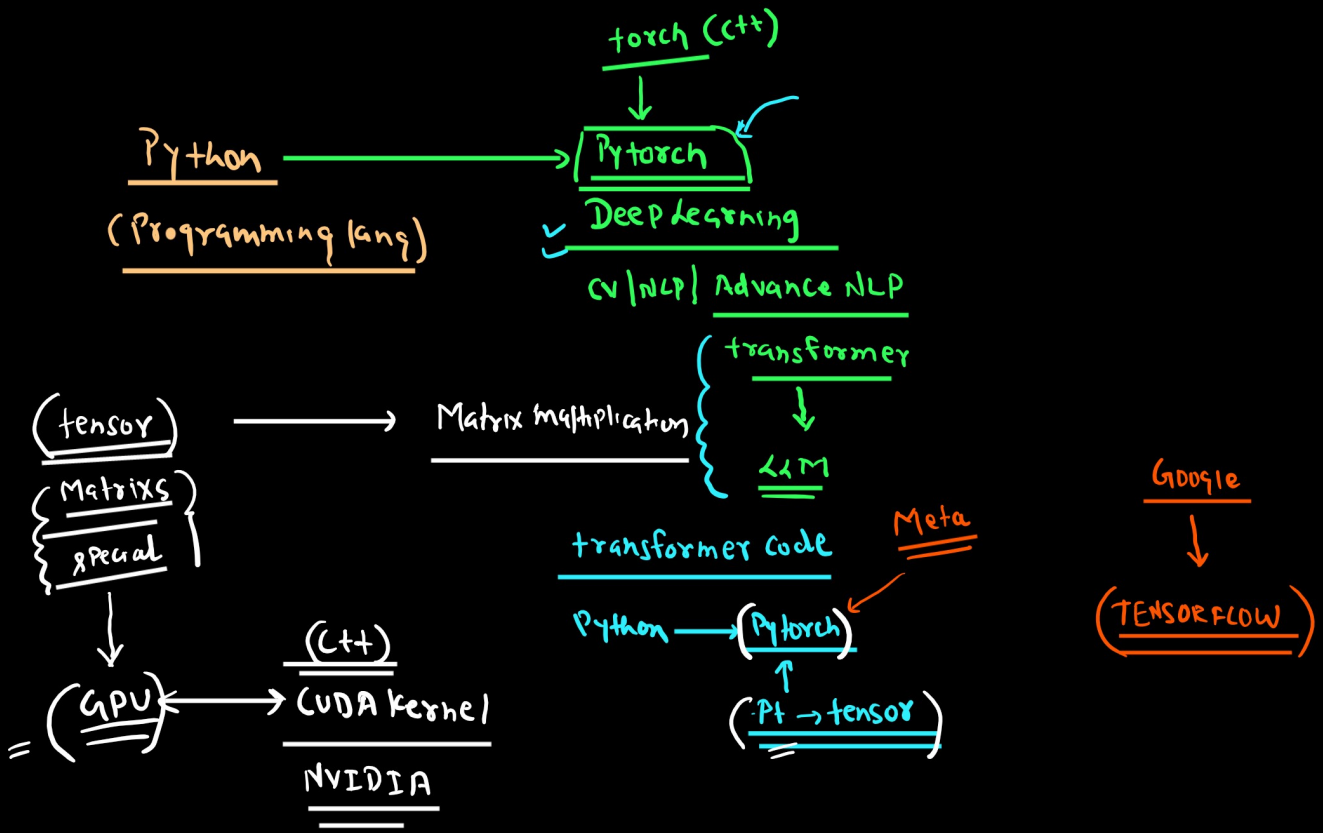


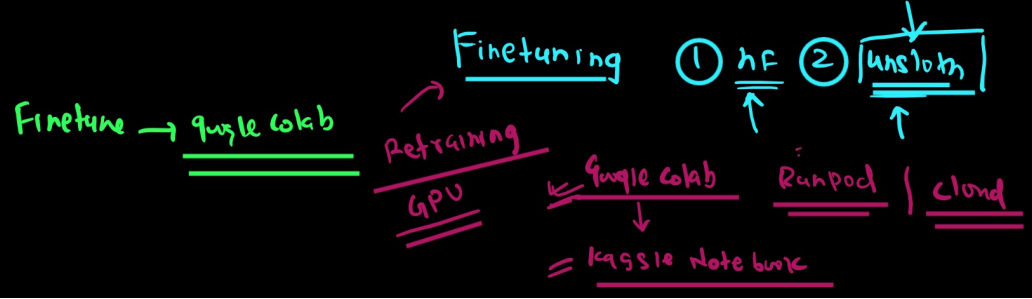
Collect-

Again we train

Retrain our model







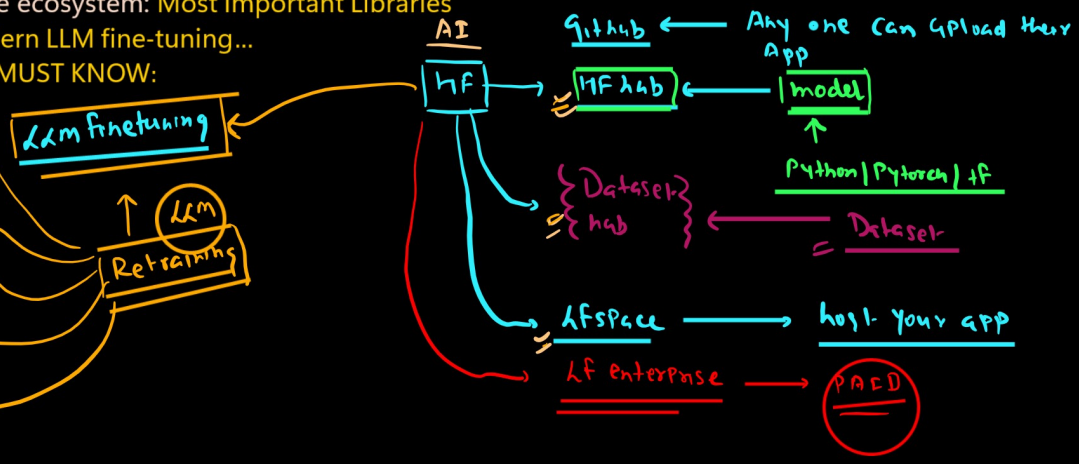
Hugging Face is an AI company that provides a complete ecosystem for AI development

Mainly including training LLMs from scratch, fine-tuning, evaluation, hosting applications.

almost the entire modern LLM fine-tuning industry workflow revolves around the Hugging Face ecosystem.

In modern Hugging Face ecosystem: Most Important Libraries
"If your goal is only modern LLM fine-tuning...
Then these libraries are MUST KNOW:

- Transformers
- Datasets
- Tokenizers
- PEFT
- Accelerate
- TRL
- Bitsandbytes
- Safetensors
- Evaluate
- Lighteval



Evaluation Matrix:

- accuracy
- F1-score
- BLEU
- ROUGE
- perplexity
- benchmark scores

evaluate | Lighteval

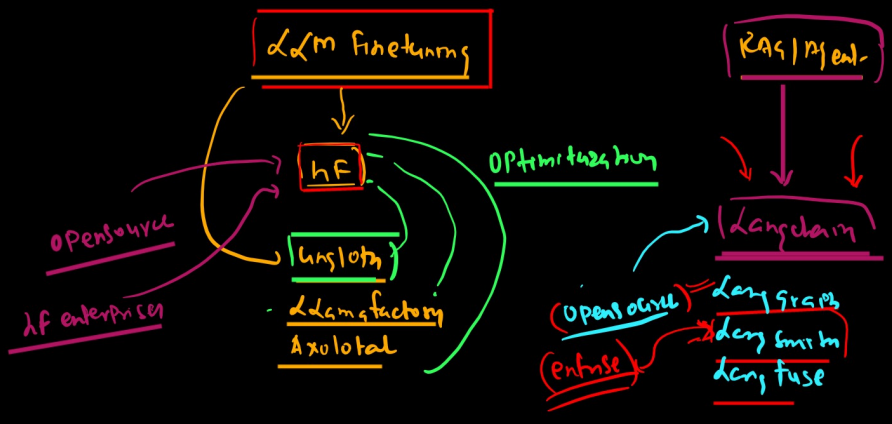
Native vs Outsider Library
"Now here is an interesting point...
Not every tool is originally built by Hugging Face."

AI COMPANY
Community
open source

Native HF Libraries

- Transformers
- Datasets
- Tokenizers
- PEFT
- Accelerate
- TRL
- Diffusers
- Evaluate
- Optimum
- Safetensors

HF Developer



Outsider but Integrated

- Bitsandbytes
- timm
- Sentence Transformers
- Argilla
- Distilabel

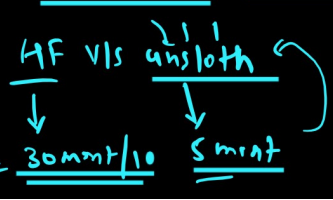
So the ecosystem is integrated, even if ownership is different.

→ Datasets Library/ Custom Dataset
→ Preprocessing / Cleaning
→ Tokenizer (Tokenized Dataset)
→ Transformers Model Loading
→ PEFT (LoRA Adapters)
→ Bitsandbytes (4-bit / Quantization)
→ Accelerate (Distributed & Multi-GPU Training)
→ Trainer / TRL (SFT, RLHF, DPO, PPO, GRPO)
→ Evaluation (Accuracy, BLEU, ROUGE, Perplexity, Benchmarks)
→ Save Checkpoints (Safetensors)
→ Push to Hugging Face Hub
→ Inference / Deployment (TGI, TEI, Inference Endpoints)
→ Monitoring & Iteration

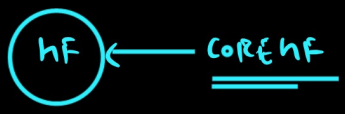
Accuracy / Quality Monitoring
Latency Monitoring
GPU / CPU Resource Monitoring
Token Usage / Cost Monitoring

Add new training data
Better LoRA adapters
RLHFDPO tuning
Hyperparameter tuning

Hugging Face is the foundation of the Finetuning ecosystem.
And Unsloth is an ultra-optimized performance layer built on top of that foundation which enables:
faster training
lower VRAM usage
longer context training
cheaper fine-tuning



Now Enters Unsloth



"Everything we discussed so far...
was the standard Hugging Face workflow.

But there was some huge issue::
slow
consumes huge VRAM
expensive

And that is exactly the problem solved by:
UNSLOTH."

Unsloth Definition

"Unsloth is an ultra-optimized LLM fine-tuning framework built on top of the Hugging Face ecosystem that enables fast LoRA/QLoRA/RLHF training with very low VRAM usage."

Main Claims of Unsloth

"Unsloth claims:

2x to 3x faster training
50–80% less VRAM usage
Ability to train large models even on free GPUs"
Real Practical Meaning

"For example:

If a Hugging Face workflow takes:

10 hours
40 GB VRAM

Then Unsloth may complete the same task in:

~5 hours
12–16 GB VRAM."
LLaMA Example

"Take the example of LLaMA 3.1 8B:

Standard HF:
24 GB VRAM

Unsloth:
only ~7–8 GB."

"The most shocking feature of Unsloth is:

LONG CONTEXT TRAINING."

20K–300K+ token training becomes possible.

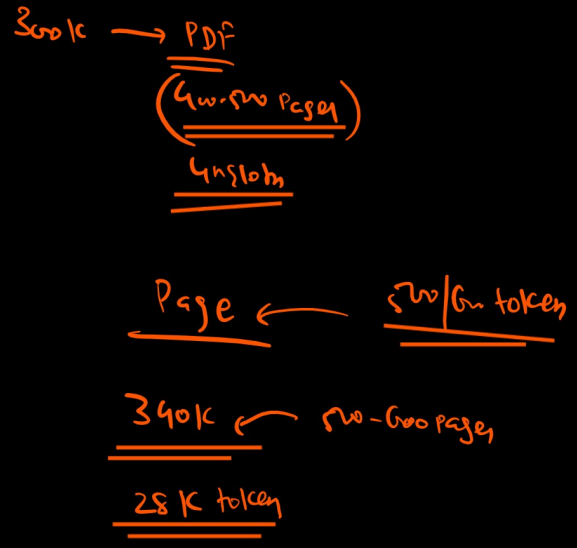
"Where standard HF gives OOM...
Unsloth can handle extremely long contexts."

GPU vs Context Length

"Example:
LLaMA 3.1 (8B)

12 GB → ~21K tokens
16 GB → ~40K
24 GB → ~78K tokens
80 GB → ~340K tokens

Whereas standard HF:
Even on an 80 GB GPU, only ~28K tokens are supported."

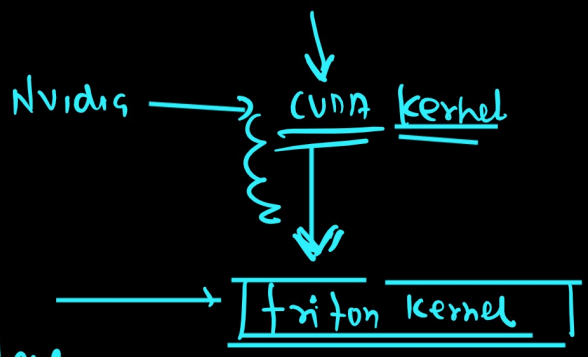


But HOW is Unsloth So Fast?

"Now the question is...

How is this magic happening?"

Answer:
Deep low-level GPU optimizations.



CUDA and Triton

"CUDA is NVIDIA's low-level GPU programming layer.

It:

runs on GPUs
is massively parallel
is extremely fast
provides low-level control

tensor

Faster

And Triton is a Python-like GPU language that generates highly optimized CUDA kernels."

Internal Optimizations

"Internally, Unsloth uses:

custom CUDA kernels
Triton kernels
fused attention
optimized forward/backward pass
smart checkpointing
Flash Attention
manual backpropagation
automatic sequence packing"

updated attention machanism

Unsloth

“The most important point:

Unsloth does not rely on approximation tricks.

It maintains exact math.

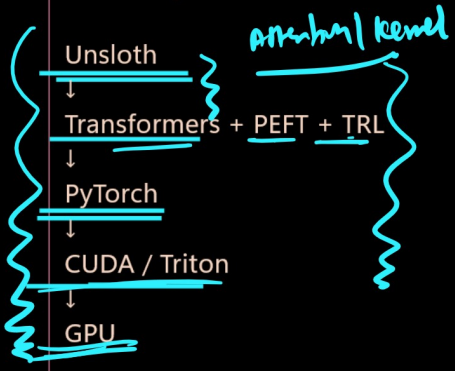
That is why:

accuracy loss is negligible
model quality does not degrade”
Architecture Flow

2.3456×8.9123=20.90518688
2.3×8.9≈20.47

“Exact math means Unsloth improves speed and memory efficiency without changing the actual mathematical correctness of the model computations.”

“Internally, the stack looks something like this:”



“This means Unsloth does not replace Hugging Face.

Instead, it works like an optimized engine on top of Hugging Face.”